

**ΣΥΣΤΗΜΑΤΑ ΑΝΑΚΤΗΣΗΣ
ΠΛΗΡΟΦΟΡΙΩΝ
Β ΦΑΣΗ**

ΙΩΑΝΝΑ ΠΑΠΑΔΑΚΑΚΗ – 3220148

ΘΕΟΦΑΝΗ ΤΣΙΟΚΑΝΗ - 3220214

Εισαγωγή

Στην παρούσα φάση της εργασίας υλοποιήθηκε ένα σύστημα ανάκτησης πληροφορίας, το οποίο σε αντίθεση με την Α' φάση όπου η ανάκτηση βασιζόταν στην λεκτική συσχέτιση των κειμένων, εστιάζει στην σημασιολογική αναπαράσταση των κειμένων. Αυτή η διαδικασία υλοποιήθηκε με την χρήση των μοντέλων τύπου Transformers, για την παραγωγή διανυσματικών αναπαραστάσεων (embeddings) και με την χρήση της βιβλιοθήκης FAISS για την αποδοτική αναζήτηση πλησιέστερων γειτόνων.

Η συλλογή που χρησιμοποιήθηκε (IR2025) περιλαμβάνει 18.316 έγγραφα καθώς και ένα σύνολο ερωτημάτων με τις αντίστοιχες σωστές απαντήσεις. Τα έγγραφα αυτά μετατράπηκαν σε διανύσματα μέσω ενός προ εκπαιδευμένου μοντέλου sentence-transformers, αποθηκεύτηκαν σε ευρετήριο FAISS και στην συνέχεια αξιολογήθηκαν για τις αποδώσεις τους μέσω του εργαλείου trec_eval. Η αξιολόγηση πραγματοποιήθηκε με βάση τις μετρικές MAP και Precision@k, με σκοπό να εκτιμηθεί η ποιότητα της ακρίβειας των ανακτηθέντων αποτελεσμάτων.

Προ επεξεργασία της συλλογής

Τα δεδομένα της συλλογής IR2025 διαβάστηκαν από το αρχείο documents.csv και αποθηκεύτηκαν σε μία λίστα (all_data) με σκοπό την συγκέντρωση όλων των εγγράφων του αρχείου. Κάθε γραμμή, μετατράπηκε σε λεξικό με κλειδιά τα πεδία της συλλογής (ID, Text). Έπειτα, δημιουργήθηκαν δύο ξεχωριστές λίστες, με σκοπό να διατηρηθούν τα κείμενα των εγγράφων καθώς και τα ids. Κατά την διαδικασία αυτή,

```
all_data = []
#διαβάζει το csv σαν dictionary ανά γραμμή
with open("IR2025/documents.csv", "r", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        all_data.append(row)

document_texts = [] # λίστα με τα κείμενα των εγγράφων
document_ids = [] # λίστα με τα id's των εγγράφων

for doc in all_data:
    text = doc["Text"].strip().replace("\n", " ").replace("\t", " ") # αφαιρεί
    document_texts.append(text) # προσθέτει στην λίστα το κείμενο
    document_ids.append(int(doc["ID"])) # προσθέτει το id στην λίστα σε μορφή int

print(f"length of documents: {len(document_texts)}") # αριθμός εγγράφων
print(f"First document: {document_texts[0][:200]}...") # εμφανίζει τους πρώτους 200 χαρακτήρες του πρώτου εγγράφου
```

εφαρμόστηκαν βασικά βήματα καθαρισμού των δεδομένων, όπως η αφαίρεση κενών χαρακτήρων, ώστε τα κείμενα να βρίσκονται σε κατάλληλη μορφή για να εισαχθούν στο μοντέλο. Τέλος εκτυπώθηκαν οι πρώτοι 200 χαρακτήρες του πρώτου εγγράφου, με σκοπό τον έλεγχο ορθότητας της διαδικασίας.

Παραγωγή Embeddings με Transformers

Χρησιμοποιήθηκε το προ-
εκπαιδευμένο μοντέλο της
βιβλιοθήκης sentence-
transformers με ονομασία all-

```
# φορτώνει το προεκπαιδευμένο μοντέλο και παράγει embeddings 384 διαστάσεων
model_name = "all-MiniLM-L6-v2"
model = SentenceTransformer(model_name)
# Υπολογίζει embeddings για όλα τα κείμενα
embeddings = model.encode(document_texts, show_progress_bar=True, batch_size=256, convert_to_numpy=True)
# κανονικοποιεί τα embeddings σε διάνυσμα μήκους 1
faiss.normalize_L2(embeddings)
# εκτυπώνει αριθμό κειμένων και την διάσταση
print(f"Embeddings shape: {embeddings.shape}")
```

MiniLM-L6-v2, το οποίο παράγει διανυσματικές αναπαραστάσεις 384 διαστάσεων και είναι εκπαιδευμένο έτσι ώστε κείμενα με σημασιολογική ομοιότητα να έχουν κοντινά διανύσματα. Πιο συγκεκριμένα, το μοντέλο, λαμβάνει ως είσοδο την λίστα που περιέχει όλα τα κείμενα των εγγραφών(document_texts) και υπολογίζει για κάθε κείμενο ένα διάνυσμα χαρακτηριστικών. Η επεξεργασία πραγματοποιείται σε παρτίδες των 256 κειμένων, ώστε να επιτυγχάνεται αποδοτική χρήση της μνήμη. Τα παραγόμενα embeddings επιστρέφονται σε έναν πίνακα numpy, το οποίο είναι κατάλληλο για την βιβλιοθήκη FAISS. Τέλος, εφαρμόζεται κανονικοποίηση L2 σε κάθε embedding, ώστε όλα τα διανύσματα να έχουν μοναδιαίο μήκος. Με αυτή τη μετατροπή, η σύγκριση βασίζεται στην σημασιολογική τους εγγύτητα.

Δημιουργία ευρετηρίου FAISS

Για την αποδοτική αναζήτηση πλησιέστερων γειτόνων, χρησιμοποιήθηκε η βιβλιοθήκη FAISS. Αρχικά προσδιορίζεται η διάσταση των embeddings, η οποία από προηγούμενο στάδιο είναι 384, έτσι ώστε

```
# διάσταση των embeddings
dimension = embeddings.shape[1]
# φτιάχνει το index του faiss
index = faiss.IndexIDMap(faiss.IndexFlatIP(dimension))
# προσθέτει τα embeddings μαζί με τα document ids
index.add_with_ids(embeddings, np.array(document_ids))
# εμφανίζει πόσα διανύσματα έχουν αποθηκευτεί στο index
print(f"Number of vectors in FAISS index: {index.ntotal}")
```

να δημιουργηθεί το ευρετήριο με σωστή διάσταση. Στη συνέχεια, κατασκευάζεται το ευρετήριο, (IndexIDMap), το οποίο βασίζεται σε δείκτη IndexFlatIP και χρησιμοποιεί το inner product ως μέτρο ομοιότητας. Η χρήση IndexIDMap επιτρέπει την συσχέτιση κάθε διανύσματος με το πραγματικό id του εγγράφου, διευκολύνοντας της ανάκτηση και την αντιστοίχιση των αποτελεσμάτων με την συλλογή. Έπειτα εισάγονται στο ευρετήριο όλα τα embeddings των εγγράφων μαζί με τα αντίστοιχα id τους και τέλος εκτυπώνεται το σύνολο των διανυσμάτων που έχουν αποθηκευτεί στο ευρετήριο, έτσι ώστε να επιβεβαιωθεί η ορθότητα της διαδικασίας.

Υλοποίηση Αναζήτησης Ερωτημάτων στο Ευρετήριο

Η συγκεκριμένη συνάρτηση υλοποιεί την αναζήτηση σημασιολογικά όμοιων εγγραφών για ένα δοσμένο ερώτημα, αξιοποιώντας τα

```
# Συνάρτηση που παίρνει το κείμενο του ερωτήματος και τα αποτελέσματα επιστροφής
def search(query, k):
    # Υπολογίζει embedding query
    query_vec = model.encode([query], convert_to_numpy=True)
    # κανονικοποίηση του query ώστε να έχει μήκος 1
    faiss.normalize_l2(query_vec)
    # Αναζητάει top-k
    scores, ids = index.search(query_vec, k)
    # Επιστρέφει λίστα (id, score, text)
    results = []
    for idx, score in zip(ids[0], scores[0]):
        doc_text = next(doc["Text"] for doc in all_data if int(doc["ID"]) == idx)
        results.append((idx, score, doc_text[:200]+"...")) # εμφανίζουμε πρώτα 200 chars
    return results
```

embeddings και το ευρετήριο FAISS, που έχουν υπολογιστεί σε προηγούμενα βήματα. Πιο συγκεκριμένα, λαμβάνει ως είσοδο το κείμενο του ερωτήματος(query) καθώς και τον αριθμό αποτελεσμάτων που θέλουμε να ανακτηθούν(k). Έπειτα, υπολογίζει το embedding

του ερωτήματος με το προ εκπαιδευμένο μοντέλο που έχει χρησιμοποιηθεί στην εργασία, ώστε το ερώτημα να αναπαρασταθεί στον ίδιο διανυσματικό χώρο με τα λοιπά έγγραφα, καθώς κι εφαρμόζεται και κανονικοποίηση μοναδιαίου μήκους, για να υπάρχει συνέπεια στην σύγκριση των διανυσμάτων. Εκτελείται αναζήτηση στο ευρετήριο FAISS, η οποία επιστρέφει τις τιμές ομοιότητας για τα k πιο κοντινά έγγραφα και τα ids των αντίστοιχων εγγραφών. Τέλος επιστρέφει την λίστα results, η οποία περιέχει τα k έγγραφα με την μεγαλύτερη σημασιολογική ομοιότητα.

Φόρτωση και Προ επεξεργασία Ερωτημάτων

Στο παρόν τμήμα κώδικα υλοποιείται η προ επεξεργασία των ερωτημάτων, με τρόπο αντίστοιχο με εκείνον που εφαρμόστηκε στην προ επεξεργασία της συλλογής των εγγραφών. Πιο συγκεκριμένα, το αρχείο queries.csv, το οποίο περιέχει τα ερωτήματα διαβάζεται και κάθε γραμμή του αποθηκεύεται στην λίστα που

```
import csv

queries_data = []
with open("IR2025/queries.csv", "r", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        queries_data.append(row)

query_ids=[]
query_texts=[]

for q in queries_data:
    text = q["Text"].strip().replace("\n", " ").replace("\t", " ")
    query_texts.append(text)
    query_ids.append((q["ID"]).strip())

print(f"length of queries : {len(query_texts)}")
print(f"First query : {query_texts[0]}")
```

έχει δημιουργηθεί (queries_data). Έπειτα, δημιουργούνται δύο ξεχωριστές λίστες(query_ids, query_texts), με σκοπό τον διαχωρισμό των αναγνωριστικών των ερωτημάτων και των αντίστοιχων κειμένων τους. Κατά την διαδικασία αυτή, εφαρμόστηκαν βασικά βήματα καθαρισμού , όπως η αφαίρεση περιττών κενών χαρακτήρων. Τέλος, εκτυπώνεται το πλήθος των ερωτημάτων που φορτώθηκαν, καθώς και το κείμενο του πρώτου ερωτήματος, με στόχο τον έλεγχο ορθότητας της διαδικασίας.

Αποθήκευση Embeddings Ερωτημάτων

Η παρούσα διαδικασία υλοποιείται με στόχο τη μείωση του υπολογιστικού κόστους και την αποφυγή επαναυπολογισμού των embeddings σε κάθε εκτέλεση του προγράμματος. Αρχικά, ορίζεται το όνομα του αρχείου(query_embedding.npy), το οποίο χρησιμοποιείται για την αποθήκευση των διανυσμάτων των ερωτημάτων.

```
import os

doc_embedding = "query_embedding.npy"

if os.path.exists(doc_embedding):
    embeddings = np.load(doc_embedding)
else:
    embeddings = model.encode(
        query_texts,
        batch_size=128,
        show_progress_bar=True,
        convert_to_numpy=True
    )
    faiss.normalize_L2(embeddings)
    np.save(doc_embedding, embeddings)

print("Query embeddings shape:", embeddings.shape)
```

Ελέγχεται εάν το αρχείο υπάρχει ήδη στον δίσκο, και στην περίπτωση που υπάρχει, τα embeddings φορτώνονται απευθείας από το αρχείο, εξοικονομώντας χρόνο και υπολογιστικούς πόρους. Αν το αρχείο δεν υπάρχει, υπολογίζονται εκ νέου τα embeddings για όλα τα ερωτήματα με το χρησιμοποιούμενο προ εκπαιδευμένο μοντέλο, εφαρμόζεται κανονικοποίηση ώστε τα διανύσματα να έχουν μοναδιαίο μήκος και στην συνέχεια αποθηκεύονται στο αρχείο για μελλοντική χρήση. Τέλος, εκτυπώνεται το πλήθος των embeddings που έχουν φορτωθεί ή παραχθεί, καθώς και την διάστασή τους, με σκοπό τον έλεγχο της ορθής εκτέλεσης της διαδικασίας.

Παραγωγή Run files για όλα τα ερωτήματα

Στο συγκεκριμένο τμήμα κώδικα, υλοποιείται η δημιουργία αρχείων αποτελεσμάτων σε μορφή συμβατή με το εργαλείο trec_eval, ώστε να γίνει η

```
def build_run_file(file_name, k, system_name="semantic-faiss"):
    with open(file_name, "w", encoding="utf-8") as f:
        for qid, qvec in zip(query_ids, embeddings):
            qvec = qvec.reshape(1, -1).astype("float32")

            scores, ids = index.search(qvec, k)

            for rank, (doc_id, score) in enumerate(zip(ids[0], scores[0]), start=1):
                if doc_id == -1:
                    continue
                f.write(f"{qid} Q0 {int(doc_id)} {rank} {float(score):.6f} {system_name}\n")

    build_run_file("run_k20.txt", 20)
    build_run_file("run_k30.txt", 30)
    build_run_file("run_k50.txt", 50)

    print("Run files created")
```

αξιολόγηση της απόδοσης του συστήματος. Αναλυτικότερα, δημιουργείται ένα αρχείο εξόδου , με το αντίστοιχο όνομα, για εγγραφή. Στην συνέχεια για κάθε ερώτημα,

λαμβάνεται το id του ερωτήματος και το αντίστοιχο διάνυσμα αναπαράστασης qvec από τις λίστες query_ids και embeddings. Το διάνυσμα μετασχηματίζεται στο κατάλληλο σχήμα και τύπο δεδομένων, για να είναι συμβατό με την βιβλιοθήκη FAISS. Έπειτα, εκτελείται αναζήτηση στο ευρετήριο FAISS από την οποία επιστρέφονται τα k πιο κοντινά έγγραφα για το συγκεκριμένο ερώτημα, μαζί με τις αντίστοιχες τιμές ομοιότητας. Τέλος, υπολογίζεται η θέση κατάταξης για κάθε αποτέλεσμα, ξεκινώντας από το 1 και καταγράφεται μία γραμμή στο αρχείο εξόδου σε μορφή trec_eval. Τέλος, καλείται η συνάρτηση τρεις φορές, προκειμένου να παραχθούν τρία διαφορετικά αρχεία αποτελεσμάτων για τιμές k = 20,30,50 τα οποία χρησιμοποιούνται στην διαδικασία αξιολόγησης.

Αξιολόγηση με trec_eval

Για την αξιολόγηση των μέτρων MAP και Precision@k (k = 5,10,15,20) χρησιμοποιήσαμε το εργαλείο trec_eval και το αρχείο qrels.txt που περιέχει τις σωστές απαντήσεις. Η μετρική Precision@k μετρά την ακρίβεια του συστήματος στα πρώτα k ανακτηθέντα έγγραφα, ενώ η MAP αποτελεί μία πιο λεπτομερή μετρική καθώς εξετάζει την συνάφεια των ανακτηθέντων εγγράφων σε συνδυασμό με την θέση στην οποία βρίσκονται. Για κάθε ένα αρχείο τρέξαμε την παρακάτω εντολή στο terminal : trec_eval qrels.txt k20.txt > results_k20.txt / trec_eval qrels.txt k30.txt > results_k30.txt / trec_eval qrels.txt k50.txt > results_k50.txt . Τα αποτελέσματα που πήραμε συνοψίζονται στον παρακάτω πίνακα συνολικά, καθώς και για κάθε run ξεχωριστά παρουσιάζονται στις παρακάτω εικόνες.

Για k=20

map	<u>all</u>	0.3129
P_5	<u>all</u>	0.6600
P_10	<u>all</u>	0.5000
P_15	<u>all</u>	0.3933
P_20	<u>all</u>	0.3500

Για k=30

<u>map</u>	<u>all</u>	0.3413
<u>P_5</u>	<u>all</u>	0.6600
<u>P_10</u>	<u>all</u>	0.5000
<u>P_15</u>	<u>all</u>	0.3933
<u>P_20</u>	<u>all</u>	0.3500

Για k=50

<u>map</u>	<u>all</u>	0.3744
<u>P_5</u>	<u>all</u>	0.6600
<u>P_10</u>	<u>all</u>	0.5000
<u>P_15</u>	<u>all</u>	0.3933
<u>P_20</u>	<u>all</u>	0.3500

K	MAP	P5	P10	P15	P20
20	0.3129	0.6600	0.5000	0.3933	0.3500
30	0.3413	0.6600	0.5000	0.3933	0.3500
50	0.3744	0.6600	0.5000	0.3933	0.3500

Παρατηρείται ότι :

- Η τιμή της μετρικής MAP αυξάνεται καθώς αυξάνεται το k. Αυτό συμβαίνει διότι, όσο αυξάνεται ο αριθμός ανακτηθέντων εγγράφων, αυξάνεται και η πιθανότητα να υπάρχουν περισσότερα συναφή έγγραφα σε χαμηλότερες θέσεις κατάταξης, με αποτέλεσμα να αυξάνεται ο μέσος όρος του precision ανά ερώτημα.
- Οι τιμές P5, P10, P15, P20 παραμένουν ίδιες για όλα τα K. Αυτό συμβαίνει διότι η μετρική Precision@k, όπως αναφέρθηκε και πάνω, εξαρτάται μόνο από τα πρώτα K

ανακτηθέντα έγγραφα. Έτσι επιστρέφεται η ίδια σειρά εγγραφών στις πρώτες 20 θέσεις, ανεξάρτητα εάν ζητάμε 20, 30 ή 50 αποτελέσματα.

- Υπάρχει υψηλότερη ακρίβεια στα πρώτα αποτελέσματα ($P5 = 0.66$ ενώ το $P20 = 0.35$). Αυτό δείχνει ότι το σύστημα καταφέρνει και ανακτά πολύ σχετικά κείμενα στις πρώτες θέσεις, γεγονός που είναι πολύ χρήσιμο για τον χρήστη που θέτει το ερώτημα.

Σύγκριση Αποτελεσμάτων Α και Β Φάσης

Πίνακας αποτελεσμάτων Α φάσης:

K	MAP	P5	P10	P15	P20
20	0.6895	0.8800	0.7300	0.7000	0.6300
30	0.7471	0.8800	0.7300	0.7000	0.6300
50	0.7701	0.8800	0.7300	0.7000	0.6300

Πίνακας αποτελεσμάτων Β φάσης:

K	MAP	P5	P10	P15	P20
20	0.3129	0.6600	0.5000	0.3933	0.3500
30	0.3413	0.6600	0.5000	0.3933	0.3500
50	0.3744	0.6600	0.5000	0.3933	0.3500

Η σύγκριση των δύο αποτελεσμάτων δείχνει ότι το σύστημα της Α' φάσης, υπερσχύει έναντι του σημασιολογικού συστήματος της Β' Φάσης σε όλες τις μετρικές αξιολόγησης. Πιο συγκεκριμένα :

- Η μετρική MAP στην Β' φάση είναι περίπου στο μισό από την αντίστοιχη τιμή της Α' Φάσης, γεγονός που δείχνει ότι συνολικά το σημασιολογικό σύστημα κατατάσσει τα σχετικά έγγραφα χαμηλότερα στην λίστα αποτελεσμάτων σε σχέση με το λεξικό μοντέλο που δημιουργήθηκε στην Α' Φάση.
- Αντίστοιχα, και στα Precision@k παρουσιάζεται μία διαφοροποίηση ως προς τις δύο διαφορετικές υλοποιήσεις, με τις τιμές στην Α' Φάση να είναι σαφώς υψηλότερες. Αυτό δείχνει ότι το λεξικό μοντέλο ανακτά σχετικά έγγραφα στις πρώτες θέσεις της κατάταξης, το οποίο είναι πολύ χρήσιμο για ένα σύστημα αναζήτησης.

Οι χαμηλότερες επιδόσεις του σημασιολογικού συστήματος μπορούν να αποδοθούν σε συνδυασμό παραγόντων. Αρχικά, η φύση της συλλογής που χρησιμοποιήθηκε και τα ερωτημάτων που περιέχει φαίνεται να ευνοεί την ακριβή λεξιλογική αντιστοίχιση όρων. Σε τέτοιου είδους δεδομένα, το λεξιλογικό μοντέλο, εκμεταλλεύεται άμεσα την συχνότητα εμφάνισης λέξεων και την άμεση ταύτιση όρων, και έτσι οδηγείται σε υψηλότερα επίπεδα ακρίβειας. Επιπλέον, το προ εκπαιδευμένο μοντέλο που χρησιμοποιείται στην Β' Φάση (all-MiniLM-L6-v2), δεν είναι εξειδικευμένο στο συγκεκριμένο θεματικό πεδίο της συλλογής. Ως αποτέλεσμα, οι παραγόμενες σημασιολογικές αναπαραστάσεις μπορεί να μην αποτυπώνουν επαρκώς τις εννοιολογικές διαφοροποιήσεις μεταξύ των όρων και των κειμένων. Τέλος, αξίζει να σημειωθεί ότι η απόδοση των συστημάτων ανάκτησης πληροφορίας, θα μπορούσε να βελτιωθεί σημαντικά, μέσω ενός συνδυασμού των δύο μοντέλων που εφαρμόστηκαν στις δύο φάσεις της εργασίας. Ένα υβριδικό σύστημα, το οποίο αξιοποιεί τόσο τη λεξιλογική ταύτιση των όρων όσο και την σημασιολογική ομοιότητα των κειμένων, θα μπορούσε να επιτύχει αποδοτικότερα αποτελέσματα.

Συμπέρασμα

Η Β' Φάση της εργασίας, στην οποία υλοποιήθηκε ένα σύστημα σημασιολογικής ανάλυσης, δεν επιτυγχάνεται καλύτερη απόδοση από την Α' Φάση, στην οποία χρησιμοποιήθηκε το μοντέλο BM2025. Το αποτέλεσμα αποδεικνύει πως, για την συγκεκριμένη συλλογή και το είδος ερωτημάτων, θα μπορούσε να χρησιμοποιηθεί ένα πιο εξειδικευμένο μοντέλο είτε να

χρησιμοποιηθεί ένας συνδυασμός μεθόδων ανάκτησης πληροφορίας, με σκοπό να βελτιωθούν οι αποδόσεις.